Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Florianópolis
2027

Ilton Pfleger Júnior

Um Hipervisor Tipo-1 Leve de Tempo Real para Sistemas Embarcados RISC-V: Projeto e Implementação em C++

Universidade Federal de Santa Catarina

Orientador: Antônio Augusto Fröhlich

Florianópolis
2027

LISTA DE ILUSTRAÇÕES

Figura 1 – Hipervisor Tipo 1. 6
Figura 2 – Hipervisor Tipo 2. 6

SUMÁRIO

1	INTRODUÇÃO	4
2	FUNDAMENTAÇÃO TEÓRICA	5
2.1	EPOS	5
2.2	Virtualização	5
2.3	Hipervisors	5
2.3.1	Tipo 1 - Bare Metal	5
2.3.2	Tipo 2	6
2.4	RISC-V	7
2.4.1	Extensões	7
2.4.2	Níveis de Privilégio	8
2.4.3	Control and Status Registers (CSRs)	8
2.5	Trap And Emulate	8
2.6	VirtIO	8
	REFERÊNCIAS	9

1 INTRODUÇÃO

Sistemas críticos estão presentes em uma ampla gama de aplicações, que vão desde dispositivos médicos e sistemas de automação industrial até plataformas automotivas e missões espaciais. Diferentemente da computação de propósito geral, esses sistemas operam sob rigorosos requisitos de tempo real, nos quais a corretude do comportamento não depende apenas da precisão lógica dos resultados produzidos, mas também do momento exato em que esses resultados são disponibilizados. Em muitos casos, atrasos mínimos ou falhas temporais podem comprometer a segurança, a confiabilidade e até mesmo a integridade física de pessoas e equipamentos.

Tradicionalmente, a garantia desse determinismo esteve associada ao uso de plataformas dedicadas, nas quais Sistemas Operacionais de Tempo Real (RTOSs) são empregados para assegurar previsibilidade temporal, isolamento entre tarefas e cumprimento de restrições rígidas de escalonamento. Entretanto, a evolução do software contemporâneo deslocou grande parte dos ecossistemas de desenvolvimento — bibliotecas, frameworks, ferramentas e serviços — para ambientes baseados em Sistemas Operacionais de Propósito Geral (GPOSs), especialmente Linux. Embora esses sistemas ofereçam grande riqueza funcional e amplo suporte da comunidade, sua adoção em contextos críticos demanda esforços adicionais de adaptação, validação e integração.

Como resultado, cresce a necessidade de consolidar, em uma mesma plataforma computacional, cargas de trabalho com requisitos distintos e frequentemente conflitantes: de um lado, aplicações críticas, sensíveis a prazos temporais e dependentes de comportamento determinístico; de outro, aplicações complexas, orientadas a funcionalidades avançadas e dependentes de ambientes generalistas.

Nesse cenário, técnicas de virtualização surgem como uma alternativa promissora, pois permitem a coexistência de múltiplos ambientes de execução sobre o mesmo hardware. Dessa forma, torna-se possível executar aplicações críticas e não críticas de maneira controlada, reduzindo custos de integração e aumentando o aproveitamento da plataforma.

Paralelamente, a arquitetura aberta RISC-V vem ganhando relevância devido à sua flexibilidade, modularidade e independência tecnológica. Seu conjunto de instruções extensível, aliado à ausência de barreiras proprietárias, favorece tanto a pesquisa acadêmica quanto o desenvolvimento industrial, especialmente em soluções customizadas e de baixo custo.

Por isso, este trabalho propõe o projeto e a implementação, em linguagem C++, de uma base de virtualização para sistemas baseados em RISC-V, com foco em previsibilidade temporal, isolamento entre cargas de trabalho e baixo custo de execução. Busca-se, assim, contribuir para a viabilização de plataformas embarcadas capazes de integrar aplicações heterogêneas sem comprometer requisitos críticos de desempenho e confiabilidade.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 EPOS

2.2 VIRTUALIZAÇÃO

A virtualização é uma técnica computacional que permite a execução simultânea e isolada de múltiplos sistemas operacionais em um mesmo hardware físico. Essa abordagem introduz uma camada de abstração entre a máquina real (Hospedeira) e os Sistemas Operacionais (SOs) (Convidados). Por meio dessa abstração, cada sistema operacional convidado possui a percepção de acesso exclusivo ao hardware, mesmo que os recursos estejam sendo compartilhados entre diferentes ambientes virtuais.

Históricamente virtualização sempre esteve ligada ao conceito de hipervisores que, por sua vez, tratam-se de um arcabolço software que serve de plataforma para essa modalidade.

2.3 HIPERVISORS

Um hipervisor, também conhecido como Monitor de Máquina Virtual (VMM), é uma plataforma que cria e executa máquinas virtuais via virtualização. Segundo Popek e Goldberg (1974) um VMM possuí três características essenciais: primeiro, devem fornecer um ambiente idêntico à máquina original para programas; segundo, os programas executados nesse ambiente, na pior das hipóteses, devem apresentar no máximo pequenas reduções de velocidade que urgem da camada de virtualização; e por último, os VMMs devem deter o contole total sobre os recursos do sistema.

Na visão de (GOLDBERG, 1973), fundamentalmente, existem dois tipos de hipervisores: 1 e 2.

2.3.1 Tipo 1 - Bare Metal

Hipervisores de tipo 1, também conhecidos como bare metal, são aqueles que executam diretamente sobre o hardware físico, sem a necessidade de um sistema operacional hospedeiro intermediário. Por operar diretamente sobre o hardware, esse tipo de hipervisor geralmente apresenta melhor desempenho e menor sobrecarga.

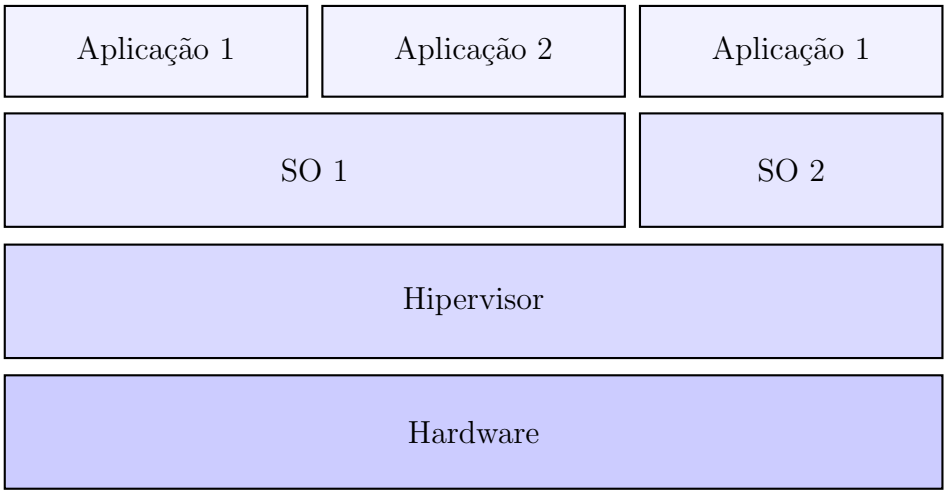


Figura 1 – Hipervisor Tipo 1.

2.3.2 Tipo 2

Diferentemente da solução bare metal, um hipervisor do tipo 2 é executado como um processo dentro de um SO hospedeiro. Porém, apesar de garantir os mesmos recursos do tipo 1, sua arquitetura impõe uma camada intermediária a mais de software, o que restringe a eficiência no acesso aos componentes físicos, gerando uma latência adicional e um menor desempenho.

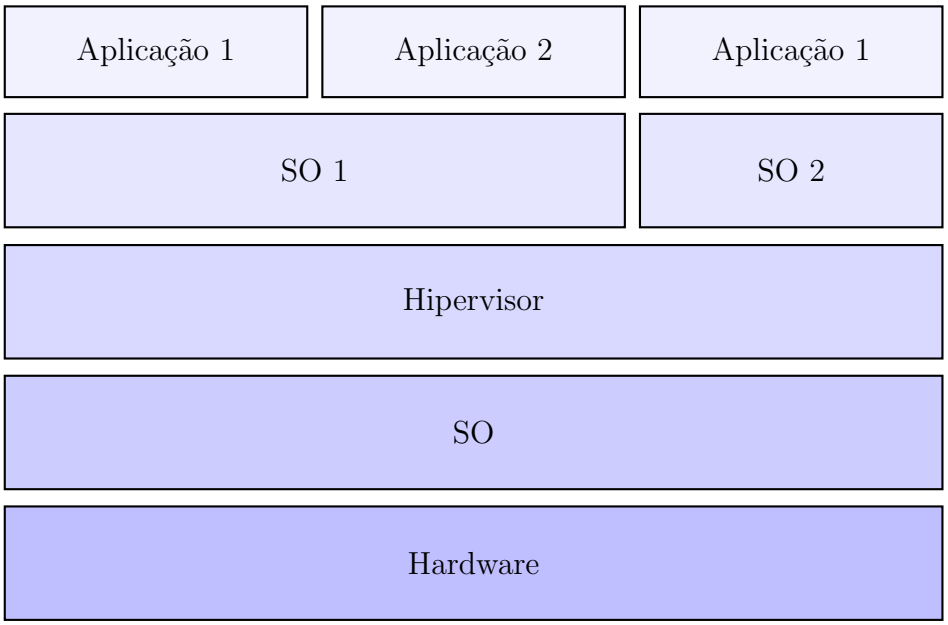


Figura 2 – Hipervisor Tipo 2.

2.4 RISC-V

O RISC-V é uma Instruction Set Architecture (ISA) baseada nos princípios da arquitetura RISC (Reduced Instruction Set Computer). Diferentemente de arquiteturas proprietárias, o RISC-V possui uma especificação aberta e livre de royalties, o que permite sua utilização tanto em contextos acadêmicos quanto em aplicações comerciais.

Essa ISA define um conjunto básico de instruções, responsável pelas funcionalidades essenciais do processador. No entanto, sua principal característica é a modularidade, permitindo que diferentes extensões sejam incorporadas conforme as necessidades da aplicação. Dessa forma, é possível adaptar a arquitetura para cenários variados, desde sistemas embarcados com restrições de recursos até sistemas de alto desempenho.

2.4.1 Extensões

I Instruções básicas para operações com números inteiros. Conforme descrito em (INTERNATIONAL, 2024), essa extensão deve estar presente em qualquer implementação RISC-V, pois define operações fundamentais de aritmética e manipulação de registradores.

Zicsr Adiciona suporte às instruções de acesso e manipulação de Control and Status Registers (CSRs).

H Adiciona suporte em hardware à virtualização. Porém, apesar de existir, é muito recente e sua implementação em hardware comercial ainda é limitada.

M Adiciona suporte à operações de multiplicação e divisão de números inteiros.

A Adiciona suporte à operações atômicas em memória.

F Adiciona suporte a números em ponto flutuante de precisão simples (32 bits).

D Adiciona suporte a números em ponto flutuante de precisão dupla (64 bits).

C Adiciona suporte à instruções comprimidas de 16 bits.

S Adiciona suporte ao modo supervisor.

G Aglomerado geral de extensões que inclui I, M, A, F, D e C.

Para o escopo deste trabalho, serão consideradas apenas as extensões Zicsr, S e G. As estratégias adotadas para mitigar a ausência da extensão H serão discutidas ao longo do texto.

2.4.2 Níveis de Privilégio

2.4.3 Control and Status Registers (CSRs)

2.5 TRAP AND EMULATE

2.6 VIRTIO

REFERÊNCIAS

GOLDBERG, R. P. Architectural principles for virtual computer systems. In: . [s.n.], 1973. Disponível em: ⟨https://apps.dtic.mil/sti/tr/pdf/AD0772809.pdf⟩.

INTERNATIONAL, R.-V. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. 2024. ⟨https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf⟩.

POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Commun. ACM*, Association for Computing Machinery, New York, NY, USA, v. 17, n. 7, p. 412–421, jul. 1974. ISSN 0001-0782. Disponível em: ⟨https://doi.org/10.1145/361011.361073⟩.